Parfait, j'ai bien tout le contexte en tête et je vais raisonner comme si je connaissais ton projet de l'intérieur.

Je récapitule très rapidement ce que j'intègre comme "contrat" entre nous, puis je te propose une suite logique pour la suite des échanges.

1. Ce que tu as déjà construit (synthèse opérationnelle)

Tu as déjà, côté PC ST (on-prem) :

1. **Ingestion**
   - Issues GitHub STM32CubeX → `raw_issues_*.json`
   - Fichiers (READMEs, Release Notes, docs) → `raw_files_*.json`
2. **Cleaning**
   - Normalisation + structuration issues → `clean_issues_*.json`
   - Normalisation + typage des fichiers (`file_type`) → `clean_files_*.json`
3. **Prétraitement V2 (enrichissement)**
   - Issues → `docs_issues_<repo>_v2.json`
     - Champs ajoutés : `is_valid`, `issue_kind`, `layer`, `severity`, `board`, `component`, etc.
   - Fichiers → `docs_files_<repo>_v2.json`
     - Champs : `file_type`, `board`, `component`, `example_name`, etc.
   - Similitude entre issues → `docs_issues_<repo>_v2_sim.json` avec `related_issue_ids`.
4. **Chunking RAG-ready**
   - Issues → `chunks_issues_<repo>_v2.json`
   - Fichiers → `chunks_files_<repo>_v2.json`
   - Chaque chunk = `text` + `metadata` riche (issue/file + heuristiques).
5. **Tests & évaluation offline**
   - Scripts de stats, cohérence V2 vs V2_sim, recherche TF-IDF.
   - Objectif : montrer que les docs/chunks sont cohérents, structurés et utiles pour un RAG.
6. **Un agent Copilot VS Code**
   - Purement pour l'ingénierie (analyse stats, documentation JSON, chunking best practices).
   - Pas dans la chaîne d'exécution.

2. Objectif sur le PC hors domaine

Sur ton PC perso / hors domaine, tu pars **des JSON déjà produits** (RAG-ready) et tu veux :

1. Construire un **pipeline RAG minimal** :
   - Indexer `chunks_*_v2.json` dans une base vectorielle (Chroma, FAISS, etc.).
   - Calculer des embeddings (modèle open source local ou API).
   - Implémenter un retrieval simple (k-NN sur embeddings).
   - Optionnel : ajouter une couche génération (LLM local ou API) pour faire du QA complet.
2. Définir et tester **plusieurs stratégies de chunking** (au moins en concept sur ton PC ST, puis en pratique sur PC hors domaine) :
   - `V1_full` (grands chunks / quasi-full docs),
   - `V2_paragraph` (ton chunking actuel par paragraphes),
   - `V2_small` (chunks plus petits),
   - `V2_sections` (pour fichiers : par section/heading).
3. Mettre en place une **évaluation expérimentale** :
   - Jeu de questions basées sur des issues et docs réelles.
   - Comparer les configurations de chunking (rappel/pertinence, taille du contexte, bruit).
   - En tirer des "bonnes pratiques" concrètes pour ton rapport de PFE.
4. Produire la **documentation** :
   - Architecture globale (data pipeline + RAG sandbox).
   - Schémas des JSON (docs_*v2, chunks*_v2).
   - Méthodologie d'évaluation.
   - Résultats + recommandations.

3. Comment je te propose d'avancer (plan de travail avec moi)

Je te propose de travailler en plusieurs étapes, que tu pourras suivre dans l'ordre ou sauter si certaines sont déjà faites.

Étape A – Définir le pipeline RAG minimal (design + squelette de code)

Objectif : obtenir un **script ou petit module Python** qui fait, pour un repo (ex. H7) :

1. Charge les chunks JSON (`chunks_issues_stm32cubeh7_v2.json`, `chunks_files_stm32cubeh7_v2.json`).
2. Construit un index vectoriel (Chroma ou FAISS).
3. Expose une fonction :

```python
def answer_question(query: str, top_k: int = 5) -> dict:
    ...
```

   qui retourne :
   - les chunks les plus proches (texte + metadata),
   - éventuellement une réponse générée (si tu as un LLM local).

Sur cette étape, je peux t'aider à :

- Choisir la techno (par ex. **Chroma + sentence-transformers**).
- Proposer une structure de code propre (modules, classes).
- Écrire un premier script complet (type `rag_sandbox.py` ou `app.py`).

Étape B – Gérer plusieurs configurations de chunking

Même si les chunks sont déjà calculés côté ST, côté PC hors domaine tu peux simuler :

- `config.rag_mode = "issues_only"`, `"files_only"`, `"mixed"`.
- `config.chunk_version = "v2_paragraph"` (actuel), et plus tard `"v2_small"`, `"v2_sections"` si tu génères d'autres JSON chunks.

Je peux t'aider à :

- Définir une **convention de nommage** claire (par ex. `chunks_issues_<repo>_<strategy>.json`).
- Coder un loader de dataset flexible, capable de charger différents fichiers selon la stratégie.
- Concevoir comment tu vas **loguer et comparer** les résultats par stratégie (même jeu de questions, même code, seuls les fichiers de chunks changent).

Étape C – Méthodologie d'évaluation et code d'éval

On pourra :

1. Définir un **jeu de questions** basé sur :
   - des issues typiques (bug HAL, config CubeMX, board NUCLEO X, etc.),
   - des cas d'usage sur les README / Release Notes (ex. "Quelle est la fonctionnalité ajoutée en v1.11.0 pour ETH sur NUCLEO-H743ZI ?").
2. Concevoir des **métriques simples mais défendables** :
   - Niveau 1 (sans LLM) : évaluer la pertinence du **retrieval** seul.
     - Pour chaque question, tu définis manuellement un ou plusieurs chunks "réponse de référence".
     - Mesurer si ces chunks sont retrouvés dans le top-k (R@k, MRR).
   - Niveau 2 (avec LLM) : juger la qualité de la réponse générée (plutôt qualitatif ou avec une grille de scoring 0-3).
3. Écrire un **script d'évaluation** :
   - Charge un `questions.json` ou `questions.csv` (question, id, réponse cible / chunks cible).
   - Teste chaque configuration de chunking et produit un rapport (tableau en Markdown ou CSV).

Je peux te fournir :

- un format concret pour le fichier de questions,
- un code Python pour calculer R@k, MRR,
- du texte pour expliquer cette méthodologie dans ton rapport.

Ét